In [8]:
```
spark
```

Out[8]:

SparkSession - hive

SparkContext

Spark UI

Version	v3.5.3
Master	yarn
AppName	Analyse_Ecom_Temps_Reel

In [10]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StructField, StringType

# 1. Configuration de la session avec le connecteur Kafka
# Note : La version du package (ex: 3.3.2) doit correspondre à la version de Spa
spark = SparkSession.builder \
    .appName("Analyse_Ecom_Temps_Reel") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.12:3
    .getOrCreate()
```

In [11]:
```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "10.132.0.3:9092") \
    .option("subscribe", "web-events") \
    .option("startingOffsets", "earliest") \
    .load()
```

```
---------------------------------------------------------------------------
AnalysisException                         Traceback (most recent call last)
Cell In[11], line 6
      1 df = spark.readStream \
      2     .format("kafka") \
      3     .option("kafka.bootstrap.servers", "10.132.0.3:9092") \
      4     .option("subscribe", "web-events") \
      5     .option("startingOffsets", "earliest") \
----> 6     .load()

File /usr/lib/spark/python/pyspark/sql/streaming/readwriter.py:304, in DataStream
Reader.load(self, path, format, schema, **options)
    302     return self._df(self._jreader.load(path))
    303 else:
--> 304     return self._df(self._jreader.load())

File /usr/lib/spark/python/lib/py4j-0.10.9.7-src.zip/py4j/java_gateway.py:1322, i
n JavaMember.__call__(self, *args)
   1316 command = proto.CALL_COMMAND_NAME +\
   1317     self.command_header +\
   1318     args_command +\
   1319     proto.END_COMMAND_PART
   1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
   1323     answer, self.gateway_client, self.target_id, self.name)
   1325 for temp_arg in temp_args:
   1326     if hasattr(temp_arg, "_detach"):

File /usr/lib/spark/python/pyspark/errors/exceptions/captured.py:185, in capture_
sql_exception.<locals>.deco(*a, **kw)
    181 converted = convert_exception(e.java_exception)
    182 if not isinstance(converted, UnknownException):
    183     # Hide where the exception came from that shows a non-Pythonic
    184     # JVM exception message.
--> 185     raise converted from None
    186 else:
    187     raise

AnalysisException: Failed to find data source: kafka. Please deploy the applicati
on as per the deployment section of Structured Streaming + Kafka Integration Guid
e.
```

In [ ]: